

CPU Scheduling Simulator

Implementation, usage, test cases, and execution evidence

1. Program overview

The C program simulates First Come First Serve (FCFS), non-preemptive Shortest Job First (SJF), and Round Robin (RR). It reads task data, prints chronological state changes and idle intervals, and reports timing statistics and average waiting time. It simulates CPU service; it does not create operating-system processes.

2. Compile and run

```
cc -std=c11 -Wall -Wextra -pedantic proj2.c -o proj2
./proj2 test1.txt FCFS
./proj2 test1.txt SJF
./proj2 test1.txt RR 5
```

Run these commands from the directory containing the source and input files. Usage: `./proj2 input_file [FCFS|RR|SJF] [time_quantum]`. Only RR takes the fourth argument, which is a positive integer quantum in milliseconds.

3. Input and scheduling rules

The first input line gives the task count (1-20), followed by one `pid arrival_time burst_time` triple per task. All fields are integers. PIDs must be unique, arrivals nonnegative, and bursts positive. Files contain numeric data only, without shell commands, headings, or EOF markers. Rows may be unsorted.

FCFS: Run tasks to completion in arrival order.

SJF: At each dispatch, choose the shortest burst among arrived, unfinished tasks, then run it to completion.

RR: Use a circular FIFO ready queue. Run at most one quantum, then return an unfinished task to its tail. Arrivals at the quantum boundary enter before the interrupted task. Ties use arrival time, then input order.

4. Waiting-time calculation

For FCFS and SJF, $\text{waiting} = \text{start} - \text{arrival}$. For RR, $\text{waiting} = \text{completion} - \text{arrival} - \text{burst}$, including all waits between slices. The average is the sum of individual waiting times divided by the number of tasks. All times are milliseconds.

Test Cases and Results

Each input was run under all three scheduling algorithms

Test 1: Assignment sample

A long first task followed by three shorter arrivals. Confirms the supplied sample results.

test1.txt: first line = 4. Tasks (PID, arrival, burst):

(0, 0, 12); (1, 2, 4); (2, 3, 1); (3, 4, 2)

Test 2: Staggered arrivals

Five overlapping tasks with varied bursts. Checks SJF selection and multiple RR turns.

test2.txt: first line = 5. Tasks (PID, arrival, burst):

(10, 0, 5); (11, 1, 3); (12, 2, 8); (13, 4, 2); (14, 6, 4)

Test 3: Idle time and tied arrivals

The CPU is idle until time 2. Simultaneous arrivals and quantum-boundary arrivals check stable ordering.

test3.txt: first line = 4. Tasks (PID, arrival, burst):

(20, 2, 6); (21, 2, 2); (22, 8, 4); (23, 10, 1)

Input	FCFS average	SJF average	RR average	RR quantum
test1.txt	9.00	7.75	5.50	5
test2.txt	6.80	4.60	7.40	3
test3.txt	3.00	1.25	2.75	2

Verification

The source compiled with C11, all requested warnings, and warnings treated as errors. All nine example runs passed. An independent millisecond-by-millisecond model matched 609 additional runs covering random and directed cases. Checks also passed for 23 invalid-input/command-line cases and an RR run with 10,001 slices. The same suite passed with AddressSanitizer and UndefinedBehaviorSanitizer enabled.

To repeat the checks, run `make test` with Python 3 installed. Complete output for all nine example runs is saved in `output/runs/`. The next pages show representative executions for all three test cases. Screenshots capture a local browser viewer displaying the actual output returned by the compiled program.

Test 1: Execution Screenshot

Assignment sample | ./proj2 test1.txt FCFS

Assignment 05 | Program execution

Actual stdout from the locally compiled C program. Exit status: 0.

```
$ ./proj2 test1.txt FCFS
```

```
FCFS Scheduling Progress:
```

```
[time 0] PID 0 NEW -> READY  
[time 0] PID 0 READY -> RUNNING  
[time 2] PID 1 NEW -> READY  
[time 3] PID 2 NEW -> READY  
[time 4] PID 3 NEW -> READY  
[time 12] PID 0 RUNNING -> COMPLETED  
[time 12] PID 1 READY -> RUNNING  
[time 16] PID 1 RUNNING -> COMPLETED  
[time 16] PID 2 READY -> RUNNING  
[time 17] PID 2 RUNNING -> COMPLETED  
[time 17] PID 3 READY -> RUNNING  
[time 19] PID 3 RUNNING -> COMPLETED
```

```
FCFS Statistical Information:
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	12	12	0
1	2	12	16	4	10
2	3	16	17	1	13
3	4	17	19	2	13

Average Waiting Time: 9.00

Figure 1. The waiting times are 0, 10, 13, and 13 ms: $(0 + 10 + 13 + 13) / 4 = 9.00$ ms.

Test 2: Execution Screenshot

Staggered arrivals | ./proj2 test2.txt SJF

Assignment 05 | Program execution

Actual stdout from the locally compiled C program. Exit status: 0.

```
$ ./proj2 test2.txt SJF
```

```
SJF Scheduling Progress:
```

```
[time 0] PID 10 NEW -> READY  
[time 0] PID 10 READY -> RUNNING  
[time 1] PID 11 NEW -> READY  
[time 2] PID 12 NEW -> READY  
[time 4] PID 13 NEW -> READY  
[time 5] PID 10 RUNNING -> COMPLETED  
[time 5] PID 13 READY -> RUNNING  
[time 6] PID 14 NEW -> READY  
[time 7] PID 13 RUNNING -> COMPLETED  
[time 7] PID 11 READY -> RUNNING  
[time 10] PID 11 RUNNING -> COMPLETED  
[time 10] PID 14 READY -> RUNNING  
[time 14] PID 14 RUNNING -> COMPLETED  
[time 14] PID 12 READY -> RUNNING  
[time 22] PID 12 RUNNING -> COMPLETED
```

```
SJF Statistical Information:
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
10	0	0	5	5	0
13	4	5	7	2	1
11	1	7	10	3	6
14	6	10	14	4	4
12	2	14	22	8	12

Average Waiting Time: 4.60

Figure 2. The dispatch order is 10, 13, 11, 14, 12. Waiting times total 23 ms, giving an average of 4.60 ms.

Test 3: Execution Screenshot

Idle time and tied arrivals | ./proj2 test3.txt RR 2

Assignment 05 | Program execution

Actual stdout from the locally compiled C program. Exit status: 0.

```
$ ./proj2 test3.txt RR 2
RR Scheduling Progress (time quantum = 2):
[time 0 -> 2] CPU IDLE
[time 2] PID 20 NEW -> READY
[time 2] PID 21 NEW -> READY
[time 2] PID 20 READY -> RUNNING
[time 4] PID 20 RUNNING -> READY (quantum expired)
[time 4] PID 21 READY -> RUNNING
[time 6] PID 21 RUNNING -> COMPLETED
[time 6] PID 20 READY -> RUNNING
[time 8] PID 22 NEW -> READY
[time 8] PID 20 RUNNING -> READY (quantum expired)
[time 8] PID 22 READY -> RUNNING
[time 10] PID 23 NEW -> READY
[time 10] PID 22 RUNNING -> READY (quantum expired)
[time 10] PID 20 READY -> RUNNING
[time 12] PID 20 RUNNING -> COMPLETED
[time 12] PID 23 READY -> RUNNING
[time 13] PID 23 RUNNING -> COMPLETED
[time 13] PID 22 READY -> RUNNING
[time 15] PID 22 RUNNING -> COMPLETED

RR Time Slices:
PID      Start Time  End Time  Running Time
20        2           4           2
21        4           6           2
20        6           8           2
22        8          10           2
20       10          12           2
23       12          13           1
22       13          15           2

RR Statistical Information:
PID      Arrival Time  Running Time  End Time  Waiting Time
20        2             6            12         4
21        2             2             6         2
22        8             4            15         3
23       10             1            13         2
Average Waiting Time: 2.75
```

Figure 3. The initial idle interval is 0-2 ms. Waiting times are 4, 2, 3, and 2 ms, averaging 2.75 ms.